Experiment No. 1

Real-World Data Collection, Cleaning and Preprocessing using Python

Dataset used: Titanic Survival Dataset

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import LabelEncoder, MinMaxScaler

# Load the Titanic dataset (built into seaborn - same data as on Kaggle)
df = sns.load_dataset('titanic')
print("Dataset loaded successfully. Shape:", df.shape)
```

```
Dataset loaded successfully. Shape: (891, 15)
```

```
print("--- First 5 rows ---")
df.head()
```

```
--- First 5 rows ---
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_ma
0	0	3	male	22.0	1	0	7.2500	S	Third	man	T
1	1	1	female	38.0	1	0	71.2833	C	First	woman	Fa
2	1	3	female	26.0	0	0	7.9250	S	Third	woman	Fa
3	1	1	female	35.0	1	0	53.1000	S	First	woman	Fa
4	0	3	male	35.0	0	0	8.0500	S	Third	man	T

Next steps: Generate code with df | New interactive sheet

```
print("--- Column info ---")
df.info()
```

```
--- Column info ---
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 15 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   survived     891 non-null    int64
 1   pclass       891 non-null    int64
 2   sex          891 non-null    object
 3   age          714 non-null    float64
 4   sibsp        891 non-null    int64
 5   parch        891 non-null    int64
 6   fare         891 non-null    float64
 7   embarked     889 non-null    object
 8   class        891 non-null    category
```

```
 9   who          891 non-null    object
 10  adult_male   891 non-null    bool
 11  deck         203 non-null    category
 12  embark_town  889 non-null    object
 13  alive        891 non-null    object
 14  alone        891 non-null    bool
dtypes: bool(2), category(2), float64(2), int64(4), object(5)
memory usage: 80.7+ KB
```

```
print("--- Basic statistics ---")
df.describe()
```

```
--- Basic statistics ---
```

	survived	pclass	age	sibsp	parch	fare
count	891.000000	891.000000	714.000000	891.000000	891.000000	891.000000
mean	0.383838	2.308642	29.699118	0.523008	0.381594	32.204208
std	0.486592	0.836071	14.526497	1.102743	0.806057	49.693429
min	0.000000	1.000000	0.420000	0.000000	0.000000	0.000000
25%	0.000000	2.000000	20.125000	0.000000	0.000000	7.910400
50%	0.000000	3.000000	28.000000	0.000000	0.000000	14.454200
75%	1.000000	3.000000	38.000000	1.000000	0.000000	31.000000
max	1.000000	3.000000	80.000000	8.000000	6.000000	512.329200

```
print("Missing values BEFORE cleaning:")
print(df.isnull().sum())

# Fill 'age' with median (middle value), not affected by outliers
df['age'] = df['age'].fillna(df['age'].median())

# Fill 'embarked' with mode (most frequent value)
df['embarked'] = df['embarked'].fillna(df['embarked'].mode()[0])

# 'deck' has too many missing values -> drop the column
df = df.drop(columns=['deck'])

# 'embark_town' duplicates info already in 'embarked' -> drop it
df = df.drop(columns=['embark_town'])

print("\nMissing values AFTER cleaning:")
print(df.isnull().sum())
```

```
Missing values BEFORE cleaning:
survived         0
pclass           0
sex              0
age            177
sibsp            0
parch            0
fare             0
embarked         2
class            0
who              0
adult_male       0
deck           688
embark_town      2
alive            0
```

```
alone            0
dtype: int64

Missing values AFTER cleaning:
survived      0
pclass        0
sex           0
age           0
sibsp         0
parch         0
fare          0
embarked      0
class         0
who           0
adult_male    0
alive         0
alone         0
dtype: int64
```

```
duplicate_count = df.duplicated().sum()
print(f"Duplicate rows found: {duplicate_count}")

df = df.drop_duplicates()
print(f"Rows remaining after removing duplicates: {len(df)}")
```

```
Duplicate rows found: 116
Rows remaining after removing duplicates: 775
```

```
le = LabelEncoder()
df['sex_encoded'] = le.fit_transform(df['sex'])         # male=1, female=0
df['alive_encoded'] = le.fit_transform(df['alive'])      # yes=1, no=0

# One-Hot Encoding: for columns with more than 2 categories
df = pd.get_dummies(df, columns=['embarked'], prefix='embarked')

print("New encoded columns added.")
df[['sex', 'sex_encoded', 'alive', 'alive_encoded']].head()
```

```
New encoded columns added.
```

	sex	sex_encoded	alive	alive_encoded
0	male	1	no	0
1	female	0	yes	1
2	female	0	yes	1
3	female	0	yes	1
4	male	1	no	0

```
def treat_outliers_iqr(data, column):
    Q1 = data[column].quantile(0.25)
    Q3 = data[column].quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    outlier_count = ((data[column] < lower_bound) | (data[column] > upper_bound)).sum()
    print(f"Column '{column}': {outlier_count} outliers found (valid range: {lower_boun

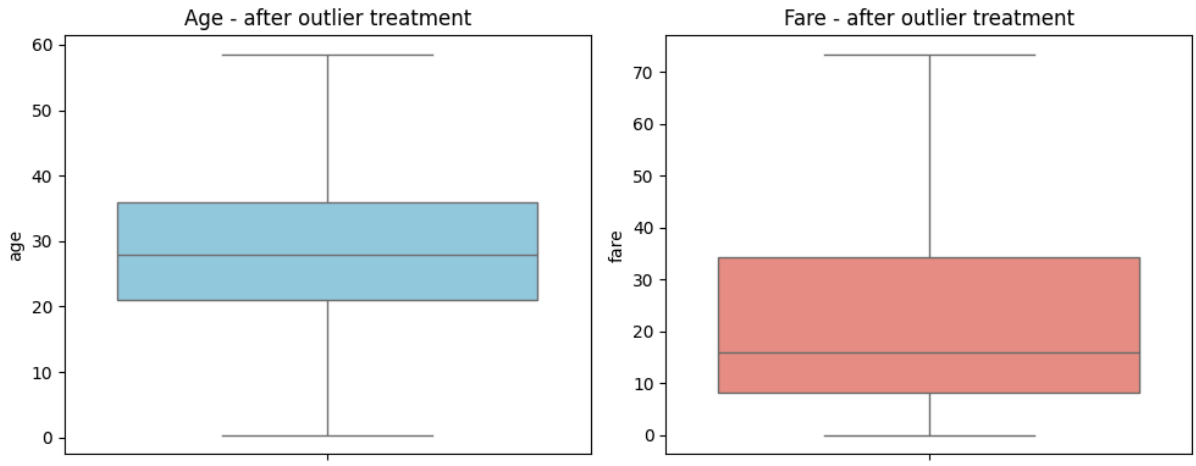
    # Cap outliers instead of deleting rows
```

```
    data[column] = np.where(data[column] > upper_bound, upper_bound, data[column])
    data[column] = np.where(data[column] < lower_bound, lower_bound, data[column])
    return data

df = treat_outliers_iqr(df, 'age')
df = treat_outliers_iqr(df, 'fare')
```

```
Column 'age': 27 outliers found (valid range: -1.50 to 58.50)
Column 'fare': 102 outliers found (valid range: -31.17 to 73.42)
```

```
fig, axes = plt.subplots(1, 2, figsize=(10, 4))
sns.boxplot(y=df['age'], ax=axes[0], color='skyblue')
axes[0].set_title('Age - after outlier treatment')
sns.boxplot(y=df['fare'], ax=axes[1], color='salmon')
axes[1].set_title('Fare - after outlier treatment')
plt.tight_layout()
plt.show()
```




```
scaler = MinMaxScaler()
df[['age_scaled', 'fare_scaled']] = scaler.fit_transform(df[['age', 'fare']])

df[['age', 'age_scaled', 'fare', 'fare_scaled']].head()
```

	age	age_scaled	fare	fare_scaled
0	22.0	0.371556	7.2500	0.098747
1	38.0	0.647039	71.2833	0.970901
2	26.0	0.440427	7.9250	0.107941
3	35.0	0.595386	53.1000	0.723239
4	35.0	0.595386	8.0500	0.109644

```
df['family_size'] = df['sibsp'] + df['parch'] + 1

# New feature 2: Age Group
df['age_group'] = pd.cut(
    df['age'], bins=[0, 12, 18, 35, 60, 100],
    labels=['Child', 'Teen', 'Young Adult', 'Adult', 'Senior']
)

# New feature 3: Fare Category
df['fare_category'] = pd.cut(
```

```
    df['fare'], bins=[-1, 10, 30, 100, 1000],
    labels=['Low', 'Medium', 'High', 'Very High']
)

df[['family_size', 'age_group', 'fare_category']].head()
```

	family_size	age_group	fare_category
0	2	Young Adult	Low
1	2	Adult	High
2	1	Young Adult	Low
3	2	Young Adult	High
4	1	Young Adult	Low

```
df.to_csv('titanic_cleaned.csv', index=False)
print("Cleaned dataset saved as 'titanic_cleaned.csv'")
print(f"Final shape: {df.shape[0]} rows, {df.shape[1]} columns")
```

```
Cleaned dataset saved as 'titanic_cleaned.csv'
Final shape: 775 rows, 22 columns
```

```
Start coding or generate with AI.
```